

Mahatma Education Society's

Pillai HOC College of Engineering and Technology, Rasayani

Department of Computer Engineering

Practical List**Subject:** System Programming & Compiler Construction**Semester:** VI

Sr. No.	Name of the Experiment	Page No.
1	Implementation of Symbolic table creation in C.	
2	Implementation of TEXT Editor with features like create, append, display and delete.	
3	Implementation of Two pass assembler.	
4	Implementation of Two pass macro processor.	
5	Implementation of Lexical Analyzer	
6	Write a program find first() and follow() sets of given grammar	
7	Implementation of LL(1) parser in C language.	
8	Implementation of Intermediate code generation phase of compiler	
9	Implementation of code generation phase of compiler.	
10	Study and implement experiments on LEX,YACC.	

H/W Requirements	RAM 512 MB, Printer, Cartridges
S/W Requirements	Turbo C, Lex/Flex, Yacc

EXPERIMENT NO: 1

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: To implement symbol table creation in C**Theory:****Symbol table:**

In computer science, a symbol table is a data structure used by a language translator such as a compiler or interpreter, where each identifier (a.k.a. symbol) in a program's source code is associated with information relating to its declaration or appearance in the source.

A symbol table may only exist during the translation process or it may be embedded in the output of that process. For example, it might be used during an interactive debugging session, or as a resource for formatting a diagnostic report during or after execution of a program.

Applications

An object file will contain a symbol table of the identifiers it contains that are externally visible. During the linking of different object files, a linker will identify and resolve these symbol references.

While reverse engineering an executable, many tools refer to the symbol table to check what addresses have been assigned to global variables and known functions. If the symbol table has been stripped or cleaned out before being converted into an executable, tools will find it harder to determine addresses or understand anything about the program.

At that time of accessing variables and allocating memory dynamically, a compiler should perform many works and as such the extended stack model requires the symbol table.

Example

Consider the following program written in C:

```
// Declare an external function
extern double bar(double x);
// Define a public function
double foo(int count)
{
```

```
double sum = 0.0;
// Sum all the values bar(1) to bar(count)
```

```
for (int i = 1; i &lt;= count; i++)
sum += bar((double) i);
return sum;
}
```

A C compiler that parses this code will contain at least the following symbol table entries:

Symbol name	Type	Scope
bar	function, double	extern
x	Double	function parameter
foo	function, double	global
count	Int	function parameter
sum	Double	block local
i	Int	for-loop statement

In addition, the symbol table will also contain entries generated by the compiler for intermediate expression values (e.g., the expression that casts the i loop variable into a double, and the return value of the call to function bar()), statement labels, and so forth

Implementation

Numerous data structures are available for implementing tables. Trees, linear lists and self-organizing lists can all be used to implement a symbol table. The symbol table is accessed by most phases of a compiler, beginning with lexical analysis, and continuing through optimization.

A compiler may use one large symbol table for all symbols or use separated hierarchical symbol tables for different scopes.

A common data structure used to implement symbol tables is the hash table. It also simplifies the classification of literals in tabular format.

If a compiler is to handle a small amount of data, then the symbol table can be implemented as an unordered list, which is easy to code, but it is only suitable for small tables only. A symbol table can be implemented in one of the following ways:

- Linear (sorted or unsorted) list
- Binary Search Tree
- Hash table

Among all, symbol tables are mostly implemented as hash tables, where the source code

symbol itself is treated as a key for the hash function and the return value is the information about the symbol.

Operations

A symbol table, either linear or hash, should provide the following operations.

insert()

This operation is more frequently used by analysis phase, i.e., the first half of the compiler where tokens are identified and names are stored in the table. This operation is used to add information in the symbol table about unique names occurring in the source code. The format or structure in which the names are stored depends upon the compiler in hand. An attribute for a symbol in the source code is the information associated with that symbol. This information contains the value, state, scope, and type about the symbol. The insert() function takes the symbol and its attributes as arguments and stores the information in the symbol table.

For example:

```
int a;
should be processed by the compiler as:
insert(a, int);
```

search()

search () operation is used to search a name in the symbol table to determine:

- if the symbol exists in the table.
- if it is declared before it is being used.
- if the name is used in the scope.
- if the symbol is initialized.
- if the symbol declared multiple times.

The format of search () function varies according to the programming language. The basic format should match the following:

search (symbol)

This method returns 0 (zero) if the symbol does not exist in the symbol table. If the symbol exists in the symbol table, it returns its attributes stored in the table.

Scope Management

A compiler maintains two types of symbol tables: a global symbol table which can be accessed by all the procedures and scope symbol tables that are created for each scope in the program.

Operations:

- Search: whether a name has been used.
- Insert: add a name.
- Delete: remove a name when its scope is closed.

A symbol table stores:

- For each type name, its type definition (eg. for the C type declaration `typedef int* mytype`, it maps the name `mytype` to a data structure that represents the type `int*`).
- For each variable name, its type. If the variable is an array, it also stores dimension information. It may also store storage class, offset in activation record etc.
- For each constant name, its type and value.
- For each function and procedure, its formal parameter list and its output type. Each formal parameter must have name, type, type of passing (by-reference or by-value), etc.

ALGORITHM:

1. Start the program
2. Define the structure of the symbol table
3. Enter the choice for performing the operations in the symbol table
4. If choice is 1, search symbol table for the symbol to be inserted. If the symbol is already present display “Duplicate Symbol”, else insert symbol and corresponding address in the symbol table
5. If choice is 2, symbols present in the symbols table are displayed
6. If choice is 3, symbol to be deleted is searched in the symbol table, if found deletes else displays “Not Found”.
7. If choice is 5, the symbol to be modified is searched in the symbol table. The label or address or both can be modified

Conclusion:

.....

.....

.....

Questions:

1. What is a symbol table?

2. Explain code optimization.

3. What are rational preprocessors?

4. What is the use of scanner generator?

5. What are the characteristics of a high-level programming language?

PROGRAM:

```

#include <stdio.h>
#include <conio.h>
#include <alloc.h>
#include <string.h>
#define null 0
int size=0;
void insert();
void del();
int search(char lab[]);
void modify();
void display();
struct symtab{
char label[10];
int addr;
struct symtab *next;};
struct symtab *first,*last;
void main(){
int op;
int y;
char la[10];
clrscr();
do{
printf("\nSYMBOL TABLE IMPLEMENTATION\n");
printf("1. INSERT\n");
printf("2. DISPLAY\n");
printf("3. DELETE\n");
printf("4. SEARCH\n");
printf("5. MODIFY\n");
printf("6. END\n");
printf("Enter your option : ");
scanf("%d",&op);
switch(op){
case 1:
insert();
display();
break;
case 2:
display();
break;
case 3:
del();
display();
break;
case 4:
printf("Enter the label to be searched : ");
scanf("%s",la);
y=search(la);

```

```

if(y==1){
printf("The label is already in the symbol Table");
}
else{
printf("The label is not found in the symbol table");
}
break;
case 5:
modify();
display();
break;
case 6:
break;
}}
while(op<6);
getch();}
void insert(){
int n;
char l[10];
printf("Enter the label : ");
scanf("%s",l);
n=search(l);
if(n==1){
printf("The label already exists. Duplicate cant be inserted\n");
}
else{
struct symtab *p;
p=malloc(sizeof(struct symtab));
strcpy(p->label,l);
printf("Enter the address : ");
scanf("%d",&p->addr);
p->next=null;
if(size==0){
first=p;
last=p;
}
else{
last->next=p;
last=p;
}
size++;
}}
void display(){
int i;
struct symtab *p;
p=first;
printf("LABEL\tADDRESS\n");
for(i=0;i<size;i++){

```



```

printf("%s\t%d\n",p->label,p->addr);
p=p->next;
}}
int search(char lab[]){
int i,flag=0;
struct symtab *p;
p=first;
for(i=0;i<size;i++){
if(strcmp(p->label,lab)==0){
flag=1;
}
p=p->next;
}
return flag;
}
void modify(){
char l[10],nl[10];
int add, choice, i, s;
struct symtab *p;
p=first;
printf("What do you want to modify?\n");
printf("1. Only the label\n");
printf("2. Only the address of a particular label\n");
printf("3. Both the label and address\n");
printf("Enter your choice : ");
scanf("%d",&choice);
switch(choice){
case 1:
printf("Enter the old label\n");
scanf("%s",l);
printf("Enter the new label\n");
scanf("%s",nl);
s=search(l);
if(s==0){
printf("NO such label");
}
else{
for(i=0;i<size;i++){
if(strcmp(p->label,l)==0){
strcpy(p->label,nl);
}
p=p->next;
}}
break;
case 2:
printf("Enter the label whose address is to modified\n");
scanf("%s",l);
printf("Enter the new address\n");

```

```

    scanf("%d",&add);
    s=search(l);
    if(s==0){
        printf("NO such label");
    }
    else{
        for(i=0;i<size;i++){
            if(strcmp(p->label,l)==0){
                p->addr=add;
            }
            p=p->next;
        }
        break;
    case 3:
        printf("Enter the old label : ");
        scanf("%s",l);
        printf("Enter the new label : ");
        scanf("%s",nl);
        printf("Enter the new address : ");
        scanf("%d",&add);
        s=search(l);
        if(s==0){
            printf("NO such label");
        }
        else{
            for(i=0;i<size;i++){
                if(strcmp(p->label,l)==0){
                    strcpy(p->label,nl);
                    p->addr=add;
                }
                p=p->next;
            }
            break;
        }
    void del(){
        int a;
        char l[10];
        struct symtab *p,*q;
        p=first;
        printf("Enter the label to be deleted\n");
        scanf("%s",l);
        a=search(l);
        if(a==0){
            printf("Label not found\n");
        }
        else{
            if(strcmp(first->label,l)==0){
                first=first->next;

```

```

}
else if(strcmp(last->label,l)==0){
q=p->next;
while(strcmp(q->label,l)!=0){
p=p->next;
q=q->next;
}
p->next=null;
last=p;
}
else{
q=p->next;
while(strcmp(q->label,l)!=0){
p=p->next;
q=q->next;
}
p->next=q->next;
}
size--;
}}

```

OUTPUT:

```

OUTPUT:
SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY
6.END
Enter your option:1
Enter the label: A
Enter the address: 10
LABEL    ADDRESS
A         10
SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY

```

1]

```

5.MODIFY
6.END
Enter your option:1
Enter the label: B
Enter the address: 11

LABEL    ADDRESS
A         10
B         11
SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY
6.END
Enter your option:2
LABEL    ADDRESS

```

2]

```

LABEL      ADDRESS
A           10
B           11

SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY
6.END
Enter your option:3
Enter the label to be deleted: A
LABEL      ADDRESS
B           11
SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY

```

3]

```

2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY
6.END
Enter your option:4
Enter the label to be searched: A
The label is not found in the symbol table.
SYMBOL TABLE IMPLEMENTATION
1.INSERT
2.DISPLAY
3.DELETE
4.SEARCH
5.MODIFY
6.END

```

4]

EXPERIMENT NO: 2

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: Implementation of TEXT Editor with features like create, append, display and delete.

Theory:

Introduction

A text editor is a tool that allows a user to create and revise documents in a computer. Though this task can be carried out in other modes, the word text editor commonly refers to the tool that does this interactively. Earlier computer documents used to be primarily plain text documents, but nowadays due to improved input-output mechanisms and file formats, a document frequently contains pictures along with texts whose appearance (script, size, colour and style) can be varied within the document. Apart from producing output of such wide variety, text editors today provide many advanced features of interactiveness and output.

Types of Text Editors

Depending on how editing is performed, and the type of output that can be generated, editors can be broadly classified as -

1. **Line Editors** - During original creation lines of text are recognised and delimited by end-of-line markers, and during subsequent revision, the line must be explicitly specified by line number or by some pattern context. eg. edlin editor in early MS-DOS systems.
2. **Stream Editors** - The idea here is similar to line editor, but the entire text is treated as a single stream of characters. Hence the location for revision cannot be specified using line numbers. Locations for revision are either specified by explicit positioning or by using pattern context. eg. sed in Unix/Linux. Line editors and stream editors are suitable for text-only documents.
3. **Screen Editors** - These allow the document to be viewed and operated upon as a two dimensional plane, of which a portion may be displayed at a time. Any portion may be specified for display and location for revision can be specified anywhere within the displayed portion. eg. vi, emacs, etc.
4. **Word Processors** - Provides additional features to basic screen editors. Usually support non-textual contents and choice of fonts, style, etc.

5. **Structure Editors** - These are editors for specific types of documents, so that the editor recognises the structure/syntax of the document being prepared and helps in maintaining that structure/syntax.

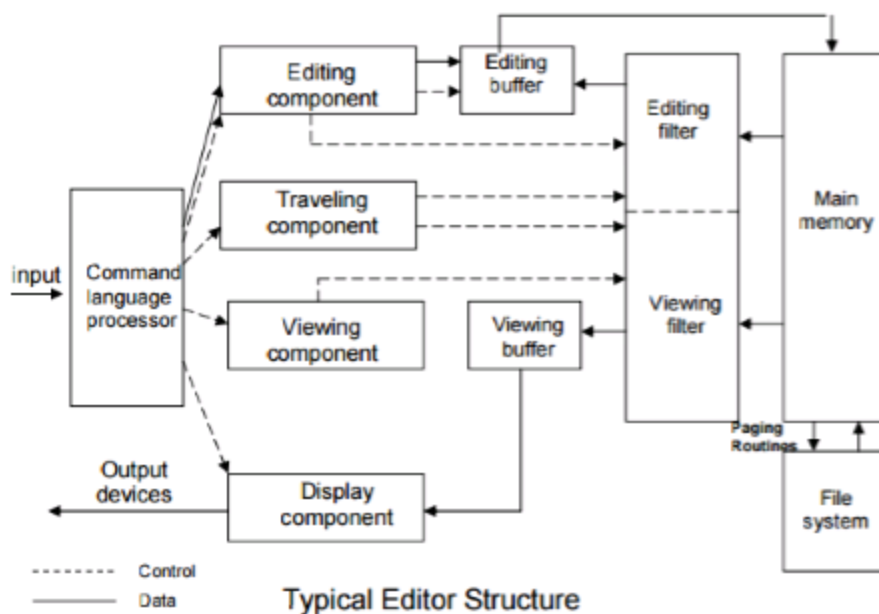
Salient Aspects of Text Editor

A text editor has to cover the following main aspects related to document creation, storage and revision -

1. Interactive user interface
2. Appropriate format for storing the document in file in secondary storage
3. Efficient transfer of information between the user interface and the file in secondary storage.

Structure of Text Editor

- Most text editors have a structure similar to that shown in the following figure.
- Command language Processor accepts command, uses semantic routines – performs functions such as editing and viewing. The semantic routines involve traveling, editing, viewing and display functions.



- Editing operations are specified explicitly by the user and display operations are specified implicitly by the editor. Traveling and viewing operations may be invoked either explicitly by the user or implicitly by the editing operations.
- In editing a document, the start of the area to be edited is determined by the current editing pointer maintained by the editing component. Editing component is a collection of modules dealing with editing tasks. Current editing pointer can be set or reset due to next paragraph, next screen, cut paragraph, paste paragraph etc.

- When editing command is issued, editing component invokes the editing filter – generates a new editing buffer – contains part of the document to be edited from current editing pointer.
- Filtering and editing may be interleaved, with no explicit editor buffer being created.
- In viewing a document, the start of the area to be viewed is determined by the current viewing pointer maintained by the viewing component.
- Viewing component is a collection of modules responsible for determining the next view.
- Current viewing pointer can be set or reset as a result of previous editing operation.
- When display needs to be updated, viewing component invokes the viewing filter – generates a new viewing buffer – contains part of the document to be viewed from current viewing pointer.
- In case of line editors – viewing buffer may contain the current line, Screen editors - viewing buffer contains a rectangular cutout of the quarter plane of the text.
- Viewing buffer is then passed to the display component of the editor, which produces a display by mapping the buffer to a rectangular subset of the screen – called a window.
- The editing and viewing buffers may be identical or may be completely disjoint. Identical – user edits the text directly on the screen.
- Disjoint – Find and Replace (For example, there are 150 lines of text, user is in 100th line, decides to change all occurrences of ‘text editor’ with ‘editor’).
- The editing and viewing buffers can also be partially overlap, or one may be completely contained in the other.
- Windows typically cover entire screen or a rectangular portion of it. May show different portions of the same file or portions of different file.
- Inter-file editing operations are possible.
- The components of the editor deal with a user document on two levels: In main memory and in the disk file system.
- Loading an entire document into main memory may be infeasible – only part is loaded – demand paging is used – uses editor paging routines.
- Documents may not be stored sequentially as a string of characters.
- Uses separate editor data structure that allows addition, deletion, and modification with a minimum of I/O and character movement.

Algorithm:

1. Display options new, open and exit and get choice.
2. If choice is 1 , call Create() function.
3. If choice is 2, call Display() function.
4. If choice is 3, call Append() function.
5. If choice is 4, call Delete() function.

6. If choice is 5, call Display() function.
7. Create()
 - 7.1 Get the file name and open it in write mode.
 - 7.2 Get the text from the user to write it.
8. Display()
 - 8.1 Get the file name from user.
 - 8.2 Check whether the file is present or not.
 - 8.2 If present then display the contents of the file.
9. Append()
 - 9.1 Get the file name from user.
 - 9.2 Check whether the file is present or not.
 - 9.3 If present then append the file by getting the text to add with the existing file.
10. Delete()
 - 10.1 Get the file name from user.
 - 10.2 Check whether the file is present or not.
 - 10.3 If present then delete the existing file.

Conclusion:

.....
.....
.....

Questions:

1. Explain structure of Editor with its salient aspects.

2. Explain different types of editor.

3. Compare MS-world with vi editor.

4. What are the functions of a text editor?

Program:

```

#include<stdio.h>
#include<conio.h>
#include<process.h>
int i,j,ch;
char fn[20],e,c;
FILE *fp1,*fp2,*fp;
void Create();
void Append();
void Copy();
void Delete();
void Display();
void main(){
do {
clrscr();
printf("\n\t\t***** TEXT EDITOR *****");
printf("\n\n\tMENU:\n\t\n");
printf("\n\t1.CREATE\n\t2.DISPLAY\n\t3.APPEND\n\t4.COPY\n\t5.DELETE\n\t6.EXIT\n");
printf("\n\tEnter your choice: ");
scanf("%d",&ch);
switch(ch){
case 1:
Create();
break;
case 2:
Display();
break;
case 3:
Append();
break;
case 4:
Copy();
break;
case 5:
Delete();
break;
case 6:
exit(0);
}
}while(1);
}
void Create()
{
fp1=fopen("temp.txt","w");
printf("\n\tEnter the text and press '.' to save\n\n\t");
while(1)
{

```

```

c=getchar();
putc(c,fp1);
if(c == '.')
{
fclose(fp1);
break;
}
}}

```

```

void Display()
{
printf("\n\tEnter the file name: ");
scanf("%s",fn);
fp1=fopen(fn,"r");
if(fp1==NULL)
{
printf("\n\tFile not found!");
goto end1;
}
while(!feof(fp1))
{
c=getc(fp1);
printf("%c",c);
}
end1:
fclose(fp1);
printf("\n\n\tPress any key to continue\n");
getch();
}

void Copy()
{
printf("\n\tEnter the new filename to copy: ");
scanf("%s",fn);
fp1=fopen("temp.txt","r");
fp2=fopen(fn,"w");
while(!feof(fp1))
{
c=getc(fp1);
putc(c,fp2);
}
fclose(fp2);
}

void Delete()
{
printf("\n\tEnter the file name: ");
scanf("%s",fn);
fp1=fopen(fn,"r");

```

```

if(fp1==NULL)
{
printf("\n\tFile not found!");
goto end2;
}
fclose(fp1);
if(remove(fn)==0)
{
printf("\n\n\tFile has been deleted successfully!");
goto end2;
}
else
printf("\n\tError!\n");
end2: printf("\n\n\tPress any key to continue\n");
getch();
}

void Append()
{
printf("\n\tEnter the file name: ");
scanf("%s",fn);
fp1=fopen(fn,"r");
if(fp1==NULL)
{
printf("\n\tFile not found!");
fclose(fp1);
goto end3;
}
while(!feof(fp1))
{
c=getc(fp1);
printf("%c",c);
}
fclose(fp1);
printf("\n\tType the text and press Ctrl+S to append.\n");
fp1=fopen(fn,"a");
while(1)
{
c=getch();
if(c==19)
goto end3;
if(c==13)
{
c='\n';
printf("\n\t");
fputc(c,fp1);
}
else

```

```
{  
printf("%c",c);  
fputc(c,fp1);  
}  
}  
end3: fclose(fp1);  
  
getch();  
}
```

Output:

```
***** TEXT EDITOR *****  
  
MENU:  
  
1.CREATE  
2.DISPLAY  
3.APPEND  
4.COPY  
5.DELETE  
6.EXIT  
  
Enter your choice: 1  
  
Enter the text and press '.' to save  
  
Siddharth.
```

```
***** TEXT EDITOR *****  
  
MENU:  
  
1.CREATE  
2.DISPLAY  
3.APPEND  
4.COPY  
5.DELETE  
6.EXIT  
  
Enter your choice: text.txt  
  
Enter the file name: Yacc is the Utility which generates the function &#39;yyparse&#39; which is indeed the Parser. Yacc describes a context free , LALR (1) grammar and supports both bottom-up and top-down parsing.The general format for the YACC file is very similar to that of the Lex file.  
  
Press any key to continue
```

```
***** TEXT EDITOR *****

MENU:

1.CREATE
2.DISPLAY
3.APPEND
4.COPY
5.DELETE
6.EXIT

Enter your choice: 3

Enter the file name: text.txt
Yacc is the Utility which generates the function &#39;yyparse&#39; which is inde
ed the Parser. Yacc describes a context free , LALR(1) grammar and supports both
bottom-up and top-down
parsing.The general format for the YACC file is very similar to that of the Lex
file.

Type the text and press Ctrl+S to append.
LL parsing is essential for compiler working
```

```
***** TEXT EDITOR *****

MENU:

1.CREATE
2.DISPLAY
3.APPEND
4.COPY
5.DELETE
6.EXIT

Enter your choice: 4

Enter the new filename to copy: new.txt_
```

```
***** TEXT EDITOR *****

MENU:

1.CREATE
2.DISPLAY
3.APPEND
4.COPY
5.DELETE
6.EXIT

Enter your choice: 5

Enter the file name: text.txt

File has been deleted successfully!

Press any key to continue
```

EXPERIMENT NO: 3

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: To write a program to implement Two pass assembler.

Theory:

Assembly Language Syntax:

Each line of a program is one of the following:

an instruction

an assembler directive (or pseudo-op)

a comment

Whitespace (between symbols) and case are ignored. Comments (beginning with “;”) are also ignored.

An instruction has the following format:

LABEL OPCODE OPERANDS ; COMMENTS

Example:

ADDR1,R1,R3

ADDR1,R1,#3

LDR6,NUMBER

BRzLOOP

Schemes of translation:**I. Two pass translation:**

It can handles forward reference easily.

Location counter(LC) processing is done in pass 1 and the symbols defined in the program are entered into symbol table.

The second pass uses this address information to generate target program.

II. Single pass translation:

Forward reference is handled by back patching.

Data structure of assembler

1. Symbol Table(ST)
2. Mnemonics table or Machine-Operation table (MOT).
3. Pseudo-opcode table (POT)
4. Literal table (LT)
5. Location counter.

Symbol table:

Symbol	Value	Location	Length	Relocation
--------	-------	----------	--------	------------

Mnemonic opcode table:

Mnemonic	Binary opcode
----------	---------------

Pseudo-opcode table (POT)

Pseudo opcode	Location of routines
---------------	----------------------

Literal table(LT)

Literal	Value	Location
---------	-------	----------

Address	Opcode	Operand
---------	--------	---------

IC:

Opcode fields:

- i. Statement class
- ii. code

Operand-2 fields

- i. Operand class:
 - S:symbol
 - L:literals
- ii. code

2-Pass Assembler:

Pass-I

I. Separate symbol, mnemonic and operand fields.

II. Build the symbol table.

I. Construct intermediate code.

Pass-II

I. Evaluate fields and generate code.

II. Process pseudo-opcodes.

Algorithm:

Pass-I

Step1: Initialize all variables:

Loc_cntr=0;

Littab_ptr=0;

Step2: Read line();

Break-words();

Step3: While (opcode!=end)

 Read line();

 Break-words();

 case1: if opcode=start or opcode=small model

 Loc_cntr=Value in operand field;

 Case2: if opcode=lable

 Insert_symtab(lable,Loc_cntr);

 Case3: if opcode=.data

 Code=search_code_MOT(opcode)

 Size=search_size_MOT(opcode)

 Loc_cntr=Loc_cntr+size;

 Generate_ic(DL,code);

 Read line();

 Break-words();

 Insert_symtab(symbol,Loc_cntr);

 Case5: else //when opcode is imperative //statement

 i.code=search_code_MOT(opcode);

 ii. Length=search_size_MOT(opcode);

 iii. Check_operand_literal();

 ❶ If yes:

 Insert_littab(literal,Loc_cntr)

 Littab_ptr++;

 ❶ If no:

Find operand entry in SYMTAB

Generate IC(IS,code)(S,entry-no);

Loc_cntr=Loc_cntr+size;

Case6:

If opcode=end

i.code=search_code_MOT(opcode);

ii.Length=search_size_MOT(opcode);

iii.Loc_cntr=Loc_cntr+size;

Step4: stop.

Conclusion:

.....
.....
.....

Questions:

1. What is the difference between one pass and two pass assembler?

2. Why do we need two pass assembler?

3. Define assembly process ?

4. What is linking and relocation?

5. Define two pass assembler?

Program:**Program for 2 pass Assemblers**

```

#include<stdio.h>
#include<string.h>
#include<conio.h>
void chk_label();
void chk_opcode();
void READ_LINE();
struct optab
{
    char code[10],objcode[10];
}myoptab[3]={
    {"LDA","00"},
    {"JMP","01"},
    {"STA","02"}
};

struct symtab{
    char symbol[10];
    int addr;
}mysymtab[10];

int startaddr,locctr,symcount=0,length;
char line[20],label[8],opcode[8],operand[8],programname[10];
/*ASSEMBLER PASS 1 */
void PASS1()
{
    FILE *input,*inter;
    input=fopen("input.txt","r");
    inter=fopen("inter.txt","w");
    printf("LOCATION LABEL\tOPERAND\tOPCODE\n");
    printf("_____");
    fgets(line,20,input);

    READ_LINE();

    if(!strcmp(opcode,"START"))
    {
        startaddr=atoi(operand);
        locctr=startaddr;
        strcpy(programname,label);
        fprintf(inter,"%s",line);
        fgets(line,20,input);
    }
    else
    {
        programname[0]='\0';
        startaddr=0;
        locctr=0;
    }
    printf("\n %d\t %s\t%s\t %s",locctr,label,opcode,operand);

```

```

while(strcmp(line,"END")!=0)
{
    READ_LINE();
    printf("\n %d\t %s \t%s\t %s",locctr,label,opcode,operand);
    if(label[0]!='\0')chk_label();
    chk_opcode();
    fprintf(inter,"%s %s %s\n",label,opcode,operand);
    fgets(line,20,input);
}

printf("\n %d\t\t%s",locctr,line);
fprintf(inter,"%s",line);

fclose(inter);
fclose(input);
}
/*Assesmbler pass 2 */
void PASS2()
{
    FILE *inter,*output;
    char record[30],part[8],value[5];/*Part array was defined as part[6] previously*/
    int currtxtlen=0,foundopcode,foundoperand,chk,operandaddr,recaddr=0;
    inter=fopen("inter.txt","r");
    output=fopen("output.txt","w");
    fgets(line,20,inter);

    READ_LINE();
    if(!strcmp(opcode,"START")) fgets(line,20,inter);
    printf("\n\nCorresponding Object code is..\n");
    printf("\nH^ %s ^ %d ^ %d ",programname,startaddr,length);
    fprintf(output,"nH^ %s ^ %d ^ %d ",programname,startaddr,length);
    recaddr=startaddr; record[0]='\0';
    while(strcmp(line,"END")!=0)
    {
        operandaddr=foundoperand=foundopcode=0;
        value[0]=part[0]='\0';
        READ_LINE();
        for(chk=0;chk<3;chk++)
        {
            if(!strcmp(opcode,myoptab[chk].code))
            {
                foundopcode=1;
                strcpy(part,myoptab[chk].objcode);
                if(operand[0]!='\0')
                {
                    for(chk=0;chk<symcount;chk++)

                    if(!strcmp(mysymtab[chk].symbol,operand))
                    {
                        itoa(mysymtab[chk].addr,value,10);
                        strcat(part,value);
                        foundoperand=1;
                    }
                }
            }
        }
    }
}

```

```

        if(!foundoperand)strcat(part,"err");
    }
}
}
if(!foundopcode)
{
    if(strcmp(opcode,"BYTE")==0 || strcmp(opcode,"WORD")||strcmp(opcode,"RESB"))
    {
        strcat(part,operand);
    }
}
if((currtxtlen+strlen(part))<=8)
/*This step was having buffer overflow issue since part[6]
was defined previously which i corrected to part[8].
Because of this first two bytes of stack are getting lost*/
{
    strcat(record,"^");
    strcat(record,part);
    currtxtlen+=strlen(part);
}
else
{
    printf("\nT^ %d ^%d %s",recaddr,currtxtlen,record);
    fprintf(output,"\nT^ %d ^%d %s",recaddr,currtxtlen,record);
    recaddr+=currtxtlen;
    currtxtlen=strlen(part);
    strcpy(record,part);
}
fgets(line,20,inter);
}
printf("\nT^ %d ^%d %s",recaddr,currtxtlen,record);
fprintf(output,"\nT^ %d ^%d %s",recaddr,currtxtlen,record);
printf("\nE^ %d\n",startaddr);
fprintf(output,"\nE^ %d\n",startaddr);
fclose(inter);
fclose(output);
}

```

```

void READ_LINE()
{
    char buff[8],word1[8],word2[8],word3[8];
    int i,j=0,count=0;
    label[0]=opcode[0]=operand[0]=word1[0]=word2[0]=word3[0]='\0';
    for(i=0;line[i]!='\0';i++)
    {
        if(line[i]!=' ')
            buff[j++]=line[i];
        else
        {
            buff[j]='\0';
            strcpy(word3,word2);
            strcpy(word2,word1);
            strcpy(word1,buff);
            j=0;

```

```

count++;
}
}
buff[j-1]='\0';
strcpy(word3,word2);
strcpy(word2,word1);
strcpy(word1,buff);
switch(count)
{
case 0:strcpy(opcode,word1);
break;
case 1:{strcpy(opcode,word2);strcpy(operand,word1);}
break;
case 2:{strcpy(label,word3);strcpy(opcode,word2);strcpy(operand,word1);}
break;
}
}
}

```

```

void chk_label()
{
int k,dupsym=0;
for(k=0;k<symcount;k++)
if(!strcmp(label,mysymtab[k].symbol))
{
mysymtab[k].addr=-1;
dupsym=1;
break;
}
if(!dupsym)
{
strcpy(mysymtab[symcount].symbol,label);
mysymtab[symcount++].addr=locctr;
}
}

```

```

void chk_opcode()
{
int k=0,found=0;
for(k=0;k<3;k++)
if(!strcmp(opcode,myoptab[k].code))
{
locctr+=3;
found=1;
break;
}
if(!found)
{
if(!strcmp(opcode,"WORD")) locctr+=3;
else if (!strcmp(opcode,"RESW"))locctr+=(3*atoi(operand));
else if(!strcmp(opcode,"RESB"))locctr+=atoi(operand);
}
}
}

```

```

int main()

```



```

{
    PASS1();
    length=locctr-startaddr;
    PASS2();
    getch();
}

```

Output:

```

C:\TURBOC3\BIN>TC
LOCATION LABEL  OPERAND  OPCODE
-----
1000  MYPGM  START    1000
1000             STA
1003  LOOP1  JMP      LOOP2
1006             LDA
1009  LOOP2  JMP      LOOP1
1012             RESB    04
1016             LDA
1019             STA
1022             JMP      LOOP1
1025             END

```

Corresponding Object code is..

```

H^ MYPGM ^ 1000 ^ 25
T^ 1000 ^8 ^02^011009
T^ 1003 ^8 00^011003
T^ 1016 ^6 04^00^02
T^ 1022 ^6 011003
E^ 1000

```

EXPERIMENT NO: 4

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: To implement a Two pass macro processor.

Theory:

- 1) **MACRO** - A macro in computer science is a rule or pattern that specifies how a certain input sequence (often a sequence of characters) should be mapped to a replacement output sequence (also often a sequence of characters) according to a defined procedure. The mapping process that instantiates (transforms) a macro use into a specific sequence is known as macro expansion. A facility for writing macros may be provided as part of a software application or as a part of a programming language. In the former case, macros are used to make tasks using the application less repetitive. In the latter case, they are a tool that allows a programmer to enable code reuse or even to design domain-specific languages.

Macros are used to make a sequence of computing instructions available to the programmer as a single program statement, making the programming task less tedious and less error-prone. Thus, they are called "macros" because a big block of code can be expanded from a small sequence of characters. Macros often allow positional or keyword parameters that dictate what the conditional assembler program generates and have been used to create entire programs or program suites according to such variables as operating system, platform or other factors. The term derives from "macro instruction", and such expansions were originally used in generating assembly language code

- 2) **MACRO Processor** - A macro processor is a program that copies a stream of text from one place to another, making a systematic set of replacements as it does so. Macro processors are often embedded in other programs, such as assemblers and compilers. Sometimes they are standalone programs that can be used to process any kind of text.

Macro processors have been used for language expansion (defining new language constructs that can be expressed in terms of existing language components), for systematic text replacements that require decision making, and for text reformatting

- 3) **MACRO in Assembly Language:** In assembly language, the term "macro" represents a more comprehensive concept than it does in some other contexts, such as in the C programming

language, where its #define directive typically is used to create short single line macros. Assembler macro instructions, like macros in PL/I and some other languages, can be lengthy "programs" by themselves, executed by interpretation by the assembler during assembly. Since macros can have 'short' names but expand to several or indeed many lines of code, they can be used to make assembly language programs appear to be far shorter, requiring fewer lines of source code, as with higher level languages. They can also be used to add higher levels of structure to assembly programs, optionally introduce embedded debugging code via parameters and other similar features.

Macro assemblers often allow macros to take parameters. Some assemblers include quite sophisticated macro languages, incorporating such high-level language elements as optional parameters, symbolic variables, conditionals, string manipulation, and arithmetic operations, all usable during the execution of a given macro, and allowing macros to save context or exchange information. Thus a macro might generate numerous assembly language instructions or data definitions, based on the macro arguments. This could be used to generate record-style data structures or "unrolled" loops, for example, or could generate entire algorithms based on complex parameters.

An organization using assembly language that has been heavily extended using such a macro suite can be considered to be working in a higher-level language, since such programmers are not working with a computer's lowest-level conceptual elements.

Underlining this point, macros were used to implement an early virtual machine in SNOBOL4 (1967), which was written in the SNOBOL Implementation Language (SIL), an assembly language for a virtual machine, which was then targeted to physical machines by transpiled to a native assembler via a macro assembler. This allowed a high degree of portability for the time.

Conclusion:

.....

Questions:

1.What is the role of Macroprocessor?

2. What is macro expansion?

3. Define Macro expansions.

4. What is Macro definition table?

5.Explain Nested micro calls.

Program:

```

#include<stdio.h>
#include<conio.h>
#include<string.h>
#include<stdlib.h>
void main()
{
    FILE *f1,*f2,*f3,*f4,*f5;
    int i,len;
    char mne[20],opnd[20],la[20],name[20],mne1[20],opnd1[20],arg[20];
    clrscr();
    f1=fopen("minp2.txt","r");
    f2=fopen("ntab2.txt","r");
    f3=fopen("dtab2.txt","r");
    f4=fopen("atab2.txt","w+");
    f5=fopen("op2.txt","w");
    fscanf(f1,"%s%s%s",la,mne,opnd);
    while(strcmp(mne,"END")!=0)
    {
        if(strcmp(mne,"MACRO")==0)
        {
            fscanf(f1,"%s%s%s",la,mne,opnd);
            while(strcmp(mne,"MEND")!=0)
                fscanf(f1,"%s%s%s",la,mne,opnd);
        }
        else
        {
            fscanf(f2,"%s",name);
            if(strcmp(mne,name)==0)
            {
                len=strlen(opnd);
                for(i=0;i<len;i++)
                {
                    if(opnd[i]!='\n')
                        fprintf(f4,"%c",opnd[i]);
                    else
                        fprintf(f4,"\n");
                }
                fseek(f2,SEEK_SET,0);
                fseek(f4,SEEK_SET,0);
                fscanf(f3,"%s%s",mne1,opnd1);
                fprintf(f5,".t%s\t%s\n",mne1,opnd);
                fscanf(f3,"%s%s",mne1,opnd1);
                while(strcmp(mne1,"MEND")!=0)
                {
                    if((opnd1[0]=='&'))
                    {
                        fscanf(f4,"%s",arg);
                        fprintf(f5,"-t%s\t%s\n",mne1,arg);
                    }
                    else
                        fprintf(f5,"-t%s\t%s\n",mne1,opnd1);
                    fscanf(f3,"%s%s",mne1,opnd1);
                }
            }
        }
    }
}

```

```

    }
    }
    else
    fprintf(f5, "%s\t%s\t%s\n", la, mne, opnd);
    }
    fscanf(f1, "%s%s%s", la, mne, opnd);
    }
    fprintf(f5, "%s\t%s\t%s\n", la, mne, opnd);
    fclose(f1);
    fclose(f2);
    fclose(f3);
    fclose(f4);
    fclose(f5);
    printf("pass2");
    getch();
}

```

Input files:

minp2.txt		
EX1	MACRO	&A,&B
-	LDA	&A
-	STA	&B
-	MEND	-
SAMPLE	START	1000

dtab2.txt	
EX1	&A,&B
LDA	&A
STA	&B
MEND	

ntab2.txt

EX1

atab2.txt

N1

N2

.	EX1	N1,N2
-	LDA	N1
N2	STA	
N1	RESW	1
N2	RESW	1
-	END	-

EXPERIMENT NO: 5

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

Aim: Implementation of Lexical Analyzer

Theory:

Lexical analysis is the first phase of a compiler. It takes modified source code from language preprocessors that are written in the form of sentences. The lexical analyzer breaks these syntaxes into a series of tokens, by removing any whitespace or comments in the source code.

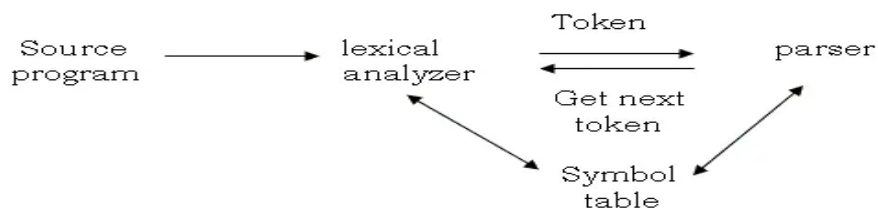
If the lexical analyzer finds a token invalid, it generates an error. The lexical analyzer works closely with the syntax analyzer. It reads character streams from the source code, checks for legal tokens, and passes the data to the syntax analyzer when it demands.

Lexical analyzer reads the characters from source code and convert it into tokens.

Tokens

Lexemes are said to be a sequence of characters (alphanumeric) in a token. There are some predefined rules for every lexeme to be identified as a valid token. These rules are defined by grammar rules, by means of a pattern. A pattern explains what can be a token, and these patterns are defined by means of regular expressions.

In programming language, keywords, constants, identifiers, strings, numbers, operators and punctuations symbols can be considered as tokens.



Different tokens or lexemes are:

- Keywords
- Identifiers
- Operators
- Constants

Take below example.

c = a + b;

After lexical analysis a symbol table is generated as given below.

Token	Type
c	identifier
=	operator
a	identifier
+	operator
b	identifier
;	separator

Conclusion:

.....

.....

.....

Questions:

1. What is Token ?

2. What are the categories of tokens?

3. What are the elements of token? Explain.

4. How does the lexical analyzer tokenize lines of codes containing spaces between strings of symbols?

Program:

```

#include<stdio.h>
#include<stdlib.h>
#include<string.h>
#include<ctype.h>

int isKeyword(char buffer[]){
char keywords[32][10] = {"auto","break","case","char","const","continue","default",
"do","double","else","enum","extern","float","for","goto",
"if","int","long","register","return","short","signed",
"sizeof","static","struct","switch","typedef","union",
"unsigned","void","volatile","while"};
int i, flag = 0;
for(i = 0; i < 32; ++i){
if(strcmp(keywords[i], buffer) == 0)
{
flag = 1;
break;
}
}
return flag;
}

int main(){
char ch, buffer[15], operators[] = "+-*/%=";
FILE *fp;
int i,j=0;
fp = fopen("program.txt","r");
if(fp == NULL){
printf("error while opening the file\n");
exit(0);
}
while((ch = fgetc(fp)) != EOF){
for(i = 0; i < 6; ++i){
if(ch == operators[i])
printf("%c is operator\n", ch);
}

if(isalnum(ch)){
buffer[j++] = ch;
}
else if((ch == ' ' || ch == '\n') && (j != 0)){
buffer[j] = '\0';
j = 0;

if(isKeyword(buffer) == 1)
printf("%s is keyword\n", buffer);
else
printf("%s is identifier\n", buffer);
}

}
fclose(fp);
return 0;
}

```

```
}
```

Output

```
include is identifier
stdioh is identifier
int is keyword
main is identifier
int is keyword
number1 is identifier
= is operator
2 is identifier
number2 is identifier
= is operator
5 is identifier
sum is identifier
sum is identifier
= is operator
number1 is identifier
+ is operator
number2 is identifier
return is keyword
0 is identifier

...Program finished with exit code 0
Press ENTER to exit console.
```

EXPERIMENT NO: 6

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: Write a program find first() and follow() sets of given grammar**THEORY:****1) FIRST:**

FIRST is applied to the R.H.S of a production rule, and tells us all the terminal symbols that can start sentences derived from that R.H.S. If a is any string of grammar symbols, let $FIRST(a)$ be the set of terminals that begin the strings derived from a . If $a \neq \epsilon$ then ϵ is also in $FIRST(a)$. It is defined as:

1. If X is a terminal **then** $First(X)$ is just X !
2. If there is a Production $X \rightarrow \epsilon$ **then** add ϵ to $first(X)$
3. If there is a Production $X \rightarrow Y_1Y_2..Y_k$ **then** add $first(Y_1Y_2..Y_k)$ to $first(X)$
4. $First(Y_1Y_2..Y_k)$ is **either**
 1. $First(Y_1)$ (if $First(Y_1)$ doesn't contain ϵ)
 2. **OR** (if $First(Y_1)$ does contain ϵ) then $First(Y_1Y_2..Y_k)$ is everything in $First(Y_1)$ <except for ϵ > as well as everything in $First(Y_2..Y_k)$
 3. If $First(Y_1)$ $First(Y_2)..First(Y_k)$ all contain ϵ **then** add ϵ to $First(Y_1Y_2..Y_k)$ as well.

2) FOLLOW:

$FOLLOW(A)$, for nonterminal A , to be the set of terminals a that can appear immediately to the right of A in some sentential form, that is, the set of terminals a such that there exists a derivation of the form $SPaAab$ for some a and b . Note that there may, at some time during the derivation, have been symbols between A and a , but if so, they derived ϵ and disappeared. If A can be the rightmost symbol in some sentential form, then $\$,$ representing the input right endmarker, is in $FOLLOW(A)$. It is used only if the current non-terminal can derive ϵ ; then we're interested in what could have followed it in a sentential form.

1. First put $\$$ (the end of input marker) in $Follow(S)$ (S is the start symbol)

2. If there is a production $A \rightarrow aBb$, (where a can be a whole string) **then** everything in $\text{FIRST}(b)$ except for ϵ is placed in $\text{FOLLOW}(B)$.
3. If there is a production $A \rightarrow aB$, **then** everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$
4. If there is a production $A \rightarrow aBb$, where $\text{FIRST}(b)$ contains ϵ , **then** everything in $\text{FOLLOW}(A)$ is in $\text{FOLLOW}(B)$

3) Example:

$$E \rightarrow TE'$$

$$E' \rightarrow +TE'$$

$$E' \rightarrow \epsilon$$

$$T \rightarrow FT'$$

$$T' \rightarrow *FT'$$

$$T' \rightarrow \epsilon$$

$$F \rightarrow (E)$$

$$F \rightarrow \text{id}$$

FIRST-

$$\text{FIRST}(E) = \{(' , \text{id})\}$$

$$\text{FIRST}(E') = \{+, \epsilon\}$$

$$\text{FIRST}(T) = \{(' , \text{id})\}$$

$$\text{FIRST}(T') = \{*, \epsilon\}$$

$$\text{FIRST}(F) = \{(' , \text{id})\}$$

FOLLOW-

$$\text{FOLLOW}(E) = \{ \$,) \}$$

$$\text{FOLLOW}(E') = \{ \$,) \}$$

$$\text{FOLLOW}(T) = \{ +, \$,) \}$$

$$\text{FOLLOW}(T') = \{ +, \$,) \}$$

$$\text{FOLLOW}(F) = \{ *, +, \$,) \}$$

Conclusion:

.....

.....

.....

Questions:

1. Find first and follow set for given grammar below

a)

$E \rightarrow T E'$	$E' \rightarrow + T E' \mid \epsilon$
$T \rightarrow F T'$	$T' \rightarrow * F T' \mid \epsilon$
$F \rightarrow (E)$	$F \rightarrow id$

b) $S \rightarrow TS \mid [S]S \mid S \mid \epsilon$

$T \rightarrow (X)$

$X \rightarrow TX \mid [X]x \mid \epsilon$

2. Explain first set and follow set with example.

3. What is difference between first() and follow()?

PROGRAM:

```

// C program to calculate the First and
// Follow sets of a given grammar

#include<stdio.h>

#include<ctype.h>

#include<string.h>

// Functions to calculate Follow

void followfirst(char, int, int);

void follow(char c);

// Function to calculate First

void findfirst(char, int, int);

int count, n = 0;

// Stores the final result
// of the First Sets

char calc_first[10][100];

// Stores the final result
// of the Follow Sets

char calc_follow[10][100];

int m = 0;

// Stores the production rules

char production[10][10];

char f[10], first[10];

int k;

char ck;

int e;

int main(int argc, char **argv)

```



```

{
    int jm = 0;

    int km = 0;

    int i, choice;

    char c, ch;

    count = 8;

    // The Input grammar

    strcpy(production[0], "E=TR");
    strcpy(production[1], "R=+TR");
    strcpy(production[2], "R=#");
    strcpy(production[3], "T=FY");
    strcpy(production[4], "Y=*FY");
    strcpy(production[5], "Y=#");
    strcpy(production[6], "F=(E)");
    strcpy(production[7], "F=i");

    int kay;

    char done[count];

    int ptr = -1;

    // Initializing the calc_first array

    for(k = 0; k < count; k++) {
        for(kay = 0; kay < 100; kay++) {
            calc_first[k][kay] = '!';
        }
    }

    int point1 = 0, point2, xxx;

```

```

for(k = 0; k < count; k++)
{
    c = production[k][0];

    point2 = 0;

    xxx = 0;

    // Checking if First of c has
    // already been calculated

    for(kay = 0; kay <= ptr; kay++)

        if(c == done[kay])

            xxx = 1;

    if (xxx == 1)

        continue;

    // Function call

    findfirst(c, 0, 0);

    ptr += 1;

    // Adding c to the calculated list

    done[ptr] = c;

    printf("\n First(%c) = { ", c);

    calc_first[point1][point2++] = c;

    // Printing the First Sets of the grammar

    for(i = 0 + jm; i < n; i++) {

        int lark = 0, chk = 0;

        for(lark = 0; lark < point2; lark++) {

            if (first[i] == calc_first[point1][lark])

                {

                    chk = 1;

```

```

        break;

    }

}

if(chk == 0)

{

    printf("%c, ", first[i]);

    calc_first[point1][point2++] = first[i];

}

}

printf("}\n");

jm = n;

point1++;

}

printf("\n");

printf("-----\n\n");

char donee[count];

ptr = -1;

// Initializing the calc_follow array

for(k = 0; k < count; k++) {

    for(kay = 0; kay < 100; kay++) {

        calc_follow[k][kay] = '!';}}

point1 = 0;

int land = 0;

for(e = 0; e < count; e++)

{

    ck = production[e][0];

    point2 = 0;

```

```

xxx = 0;

// Checking if Follow of ck

// has already been calculated

for(kay = 0; kay <= ptr; kay++)

    if(ck == donee[kay])

        xxx = 1;

if (xxx == 1)

    continue;

land += 1;

// Function call

follow(ck);

ptr += 1;

// Adding ck to the calculated list

donee[ptr] = ck;

printf(" Follow(%c) = { ", ck);

calc_follow[point1][point2++] = ck;

// Printing the Follow Sets of the grammar

for(i = 0 + km; i < m; i++) {

    int lark = 0, chk = 0;

    for(lark = 0; lark < point2; lark++)

    {

        if (f[i] == calc_follow[point1][lark])

        {

            chk = 1;

            break;

        }

    }

}

```

```

    if(chk == 0)
    {
        printf("%c, ", f[i]);

        calc_follow[point1][point2++] = f[i];
    }
}

printf(" }\n\n");

km = m;

point1++;

}

}

void follow(char c)
{
    int i, j;

    // Adding "$" to the follow

    // set of the start symbol

    if(production[0][0] == c) {
        f[m++] = '$';
    }

    for(i = 0; i < 10; i++)
    {
        for(j = 2; j < 10; j++)
        {
            if(production[i][j] == c)
            {
                if(production[i][j+1] != '\0')
                {

```

```

        // Calculate the first of the next

        // Non-Terminal in the production

        followfirst(production[i][j+1], i, (j+2));

    }

    if(production[i][j+1]!='\0' && c!=production[i][0])

    {

        // Calculate the follow of the Non-Terminal

        // in the L.H.S. of the production

        follow(production[i][0]);

    }

}

}

}

}

```

```

void findfirst(char c, int q1, int q2)

```

```

{

    int j;

    // The case where we

    // encounter a Terminal

    if(!(isupper(c))) {

        first[n++] = c;

    }

    for(j = 0; j < count; j++)

    {

        if(production[j][0] == c)

        {

```

```

if(production[j][2] == '#')
{
    if(production[q1][q2] == '\0')
        first[n++] = '#';
    else if(production[q1][q2] != '\0'
        && (q1 != 0 || q2 != 0))
    {
        // Recursion to calculate First of New
        // Non-Terminal we encounter after epsilon
        findfirst(production[q1][q2], q1, (q2+1));
    }
    else
        first[n++] = '#';
}
else if(!isupper(production[j][2]))
{
    first[n++] = production[j][2];
}
else
{
    // Recursion to calculate First of
    // New Non-Terminal we encounter
    // at the beginning
    findfirst(production[j][2], j, 3);
}
}
}

```

```

}

void followfirst(char c, int c1, int c2)
{
    int k;

    // The case where we encounter
    // a Terminal
    if(!(isupper(c)))
        f[m++] = c;
    else
    {
        int i = 0, j = 1;
        for(i = 0; i < count; i++)
        {
            if(calc_first[i][0] == c)
                break;
        }

        //Including the First set of the
        // Non-Terminal in the Follow of
        // the original query
        while(calc_first[i][j] != '#')
        {
            if(calc_first[i][j] != '#')
            {
                f[m++] = calc_first[i][j];
            }
        }
        else
        {

```



```
if(production[c1][c2] == '\0')
{
    // Case where we reach the
    // end of a production
    follow(production[c1][0]);
}
else
{
    // Recursion to the next symbol
    // in case we encounter a "#"
    followfirst(production[c1][c2], c1, c2+1);
}
}
j++;
}
}
```

OUTPUT:

```
First(E) = { (, i, }  
First(R) = { +, #, }  
First(T) = { (, i, }  
First(Y) = { *, #, }  
First(F) = { (, i, }
```

```
-----  
Follow(E) = { $, ), }  
Follow(R) = { $, ), }  
Follow(T) = { +, $, ), }  
Follow(Y) = { +, $, ), }  
Follow(F) = { *, +, $, ), }
```

```
...Program finished with exit code 0  
Press ENTER to exit console.█
```

EXPERIMENT NO: 7

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: To implement a LL(1) parser in C language.**Theory:****Parsing :**

A parser is a compiler or interpreter component that breaks data into smaller elements for easy translation into another language. A parser takes input in the form of a sequence of tokens or program instructions and usually builds a data structure in the form of a parse tree or an abstract syntax tree. In the compiler model, the parser obtains a string of tokens from the lexical analyzer, and verifies that the string can be generated by the grammar for the source language. The parser returns any syntax error for the source language. It collects sufficient number of tokens and builds a parse tree.

LL(1) Parser: Predictive parsers can be constructed for LL(1) grammar, the first 'L' stands for scanning the input from left to right, the second 'L' stands for leftmost derivation and '1' for using one input symbol look ahead at each step to make parsing action decisions.

A grammar G is LL(1) if $A \rightarrow \alpha \mid \beta$ are two distinct productions of G . For no terminal, both α and β derive strings beginning with a . At most one of α and β can derive empty string. If $\beta \rightarrow \epsilon$, then α does not derive any string beginning with a terminal in $\text{FOLLOW}(A)$.

ALGORITHM:

1. Open the input file in read format.
2. Read each line until the end of file.
3. Separate the expression and result.
4. Separate operation and operator from the expression.
5. Print the result.

6. Close the file.

Enter the no. of co-ordinates

2

Enter the productions in a grammar

$S \rightarrow CC$

$C \rightarrow eC \mid d$

First pos FIRS[S] = ed

FIRS[C] = ed

Follow pos FOLLOW[S] = \$

FOLLOW[C] = ed\$

$M[S, e] = S \rightarrow CC$

$M[S, d] = S \rightarrow CC$

$M[C, e] = C \rightarrow eC$

$M[C, d] = C \rightarrow d$

Conclusion:

Questions:

1. What is right-most derivation? Give example .

2. What is left-most derivation? Give example

3. Explain canonical derivations.

4. What is left-sentential form?

5. What is right-sentential form?

PROGRAM:

```
#include<stdio.h>

#include<stdlib.h>

#include<string.h>

int noT,noNT;

char STK[20];

int TOP = -1;

typedef struct rule
{
    char lhs;

    char rhs[10];

    int num;
}rule;

struct rule rules[15];

typedef struct frst
{
    char c;

    char set[30];

    int len;
}frst;

struct frst firsts[10];

typedef struct follow
{
    char c;

    char set[30];

    int len;

    int visited;
```

```
}follow;

struct follow follows[10];

int isNonTerminal(char c,int noRules)
{
    int i=0;
    for(i;i<noRules;i++)
        if(rules[i].lhs==c)
            return 1;
    return 0;
}

void push(char c)
{
    STK[++TOP]=c;
}

void pop()
{
    STK[TOP--]='\0';
}

void printStack()
{
    int i;
    printf("\nTOP=%d\n",TOP);
    for(i=0;i<20;i++)
    {
        if(STK[i]=='\0')
            break;
        printf("%c",STK[i]);
    }
}
```

```

    }
}

int inarr(char* arr,char c,int noRules)
{
    int i;
    for(i=0;i<noRules;i++)
        if(arr[i]==c)
            return 1;
    return 0;
}

int isInFirSet(char NT, char ch,int noNT)
{
    int i=0;
    int j=0;
    for(i=0;i<noNT;i++)
        if(firsts[i].c==NT)
            break;
    for(j=0;j<=firsts[i].len;j++)
        if(firsts[i].set[j]==ch)
            return 1;
    return 0;
}

int isInFolSet(char NT, char ch,int noNT)
{
    int i=0;
    int j=0;
    for(i=0;i<noNT;i++)

```



```

        if(follows[i].c==NT)
            break;
    for(j=0;j<=follows[i].len;j++)
        if(follows[i].set[j]==ch)
            return 1;

    return 0;
}

void copyfirst(char NT1,char NT2,int noNT)
{
    int i,j;

    char ch;

    for(i=0;i<noNT;i++)
        if(firsts[i].c==NT1)
            break;

    for(j=0;j<noNT;j++)
        if(firsts[j].c==NT2)
            break;

    int ind;

    int k;

    for(k=0;k<firsts[j].len;k++)
    {
        ind = firsts[i].len;

        ch = firsts[j].set[k];

        if(!isInFirSet(NT1,ch,noNT)&&ch!='e')
        {
            firsts[i].set[ind]=ch;

            firsts[i].len++;
        }
    }
}

```

```

        }

    }

}

void copyfollow(char NT1,char NT2,int noNT)
{
    printf("\n");

    int i,j;

    char ch;

    for(i=0;i<noNT;i++)

        if(follows[i].c==NT1)

            break;

    for(j=0;j<noNT;j++)

        if(follows[j].c==NT2)

            break;

    int ind;

    int k;

    for(k=0;k<follows[j].len;k++)

    {

        ind = follows[i].len;

        ch = follows[j].set[k];

        if(!isInFolSet(NT1,ch,noNT)&&ch!='e')

        {

            follows[i].set[ind]=ch;

            follows[i].len++;

        }

    }

}

```

```

void copyfollowfirst(char NT1,char NT2,int noNT)
{
    int i,j;

    char ch;

    for(i=0;i<noNT;i++)
        if(follows[i].c==NT1)
            break;

    for(j=0;j<noNT;j++)
        if(firsts[j].c==NT2)
            break;

    int ind;

    int k;

    for(k=0;k<firsts[j].len;k++)
    {
        ind = follows[i].len;

        ch = firsts[j].set[k];

        if(!isInFolSet(NT1,ch,noNT)&&ch!='e')
        {
            follows[i].set[ind]=ch;

            follows[i].len++;

        }
    }
}

void copyfollowfollow(char NT1,char NT2,int noNT)
{
    int i,j;

    char ch;

```

```

for(i=0;i<noNT;i++)

    if(follows[i].c==NT1)

        break;

for(j=0;j<noNT;j++)

    if(follows[j].c==NT2)

        break;

int ind;

int k;

for(k=0;k<follows[j].len;k++)

{

    ind = follows[i].len;

    ch = follows[j].set[k];

    if(!isInFolSet(NT1,ch,noNT)&&ch!='e')

    {

        follows[i].set[ind]=ch;

        follows[i].len++;

    }

}

}

void add2first(char NT1,int noRules,int noNT,int rhslen,int ruleno)

{

    rhslen = rhslen+1;

    //printf("\nNT=%c,ruleno=%d,rhslen=%d\t",NT1,ruleno,rhslen);

    int i=0,k=0;

    int j,index;

    char NT2;

    for(k=0;k<rhslen;)

```

```

{

    char NT2=rules[ruleno].rhs[k];

    if(!isNonTerminal(NT2,noRules))

    {

        //add that terminal to NT1

        for(j=0;j<noNT;j++)

            if(firsts[j].c==NT1)

                break;

        if(!isInFirSet(NT1,NT2,noNT))

        {

            index = firsts[j].len;

            firsts[j].set[index]=NT2;

            firsts[j].len++;

            return;

        }

        //printf("\n");

    }

    else if(isNonTerminal(NT2,noRules))

    {

        int temp=0;

        for(j=0;j<noRules;j++)

        {

            if((rules[j].lhs==NT2)&&isNonTerminal(rules[j].rhs[0],noRules))

            {

                temp = 1;

                //printf("\n%c\t%s",rules[j].lhs,rules[j].rhs);

                add2first(NT2,noRules,noNT,rules[j].num,j);

```

```

        copyfirst(NT1,NT2,noRules);

    }

}

if(temp==0)

{

    copyfirst(NT1,NT2,noRules);

}

//printf("\n");

}

if((k==rhslen-1)&&isInFirSet(NT2,'e',noNT))

{

    //printf("\nk=%d,NT1=%c,NT2=%c\n",k,NT1,NT2);

    int m;

    int index;

    for(m=0;m<noNT;m++)

        if(firsts[m].c==NT1)

            break;

    if(!isInFirSet(NT1,'e',noNT))

    {

        index = firsts[m].len;

        firsts[m].set[index]='e';

        firsts[m].len++;

    }

    return;

}

else if(isInFirSet(NT2,'e',noNT)&&(isNonTerminal(NT2,noRules)))

{ //printf("JJJJ\n");

```

```

        k++;}

    else { //printf("MMMMM\n");

        return;}

}

//printf("\n");

}

```

```
void add2follow(char ch,char NT1,int noNT)
```

```

{

    int j,index;

    for(j=0;j<noNT;j++)

        if(follows[j].c==NT1)

            break;

    if(!isInFolSet(NT1,ch,noNT))

    {

        index = follows[j].len;

        follows[j].set[index]=ch;

        follows[j].len++;

    }

}

```

```
int charIndexT(char c,char* term,int noT)
```

```

{

    int i;

    if(c=='\0')

        return -1;

    for(i=0;i<noT;i++)

    {

```

```

        if(c==term[i])
            return i;
    }
    return -1;
}

int charIndexNT(char c,char* nonTerm,int noNT)
{
    int i;
    if(c=='\0')
        return -1;
    for(i=0;i<noNT;i++)
    {
        if(c==nonTerm[i])
            return i;
    }
}

void printTable(int TABLE[noNT][noT],char* term,char *nonTerm)
{
    printf("\nParsing Table\n\n");
    int i,j;
    for(i=0;i<noT;i++)
        printf("\t%c",term[i]);
    printf("\n");
    for(i=0;i<noNT;i++)
    {
        printf("%c\t",nonTerm[i]);
        for(j=0;j<noT;j++)

```



```

        printf("%d\t",TABLE[i][j]);

    printf("\n");

}

}

void findfollow(int NT1,int noRules,int noNT)
{
    int i,j,k,x,m;
    for(i=0;i<noNT;i++)
        if(follows[i].c==NT1)
        {
            if(follows[i].visited==1)
            {
                return;
            }
            else
            {
                follows[i].visited=1;
                break;
            }
        }
    for(j=0;j<noRules;j++)
    {
        char next;

        for(k=0;k<=rules[j].num;k++)
        {
            if(k==rules[j].num)
            {

```

```

if(rules[j].rhs[k]==NT1)
{
    findfollow(rules[j].lhs,noRules,noNT);
    copyfollowfollow(NT1,rules[j].lhs,noNT);
}
}
else if(rules[j].rhs[k]==NT1)
{
    x = k;
    next = rules[j].rhs[++x];
    while(x<(rules[j].num+1))
    {
        int temp;

        if(!isNonTerminal(next,noRules))
        {
            add2follow(next,NT1,noNT);
            break;
        }
        else if(isNonTerminal(next,noRules))
        {
            copyfollowfirst(NT1,next,noNT);
            if(isInFirSet(next,'e',noNT))
            {
                if(x==rules[j].num)
                {
                    findfollow(rules[j].lhs,noRules,noNT);
                    copyfollowfollow(NT1,rules[j].lhs,noNT);
                }
            }
        }
    }
}
}

```

```

        }
        else
            next = rules[j].rhs[x+1];
    }
}
x++;
}
}
}
}

int main()
{
    rules[0].lhs = 'E';
    strncpy(rules[0].rhs,"TD",2);
    rules[0].num = 1;
    rules[1].lhs = 'D';
    strncpy(rules[1].rhs,"+TD",3);
    rules[1].num = 2;
    rules[2].lhs = 'D';
    strncpy(rules[2].rhs,"e",1);
    rules[2].num = 0;
    rules[3].lhs = 'T';
    strncpy(rules[3].rhs,"FU",2);
    rules[3].num = 1;
    rules[4].lhs = 'U';
    strncpy(rules[4].rhs,"*FU",3);

```

```

rules[4].num = 2;

rules[5].lhs = 'U';

strncpy(rules[5].rhs,"e",1);

rules[5].num = 0;

rules[6].lhs = 'F';

strncpy(rules[6].rhs,"(E)",3);

rules[6].num = 2;

rules[7].lhs = 'F';

strncpy(rules[7].rhs,"i",1);

rules[7].num = 0;


int i,j,k=0;

int noRules=8;

char NT,NT1,NT2;

printf("Rules. . .\n");

for(i=0;i<noRules;i++)

{

    printf("%c\t->\t",rules[i].lhs);

    for(j=0;j<(rules[i].num)+1;j++)

        printf("%c",rules[i].rhs[j]);

    printf("\t%d\n",rules[i].num);

}

//nonTerminal calc

char arr[noRules];

char ch;

for(i=0;i<noRules;)

{

```

```

        ch = rules[i].lhs;

        if(!inarr(arr,ch,noRules))

            arr[k++]=ch;

        else if(inarr(arr,ch,noRules))

            i++;

    }

    printf("\n\n\n");

    for(i=0;i<k;i++)

    {

        firsts[i].c=arr[i];

        follows[i].c=arr[i];

    }

    int index;

    noNT = k;

    char nonTerm[k];

    for(i=0;i<k;i++)

        nonTerm[i]=arr[i];

    //term calc

    char term[40];

    k=0;

    for(i=0;i<noRules;i++)

    {

        int len;

        len = rules[i].num+1;

        for(j=0;j<=len;j++)

        {

            ch = rules[i].rhs[j];

```

```

        if(!isNonTerminal(ch,noRules))
        {
            if((!inarr(term,ch,noRules))&&ch!='\0'&&ch!='e')
            {
                term[k++]=ch;
            }
        }
    }

    term[k++]='$';

    noT = k;

    //printing non term and term. . .

    printf("Terminals are. . .\n");

    for(i=0;i<noT;i++)

        printf("%c\t",term[i]);

    printf("\nNon-Terminals are. . .\n");

    for(i=0;i<noNT;i++)

        printf("%c\t",nonTerm[i]);

    //first set calculation

    printf("\n\n");

    for(i=0;i<noRules;i++)
    {

        char first=0;

        char second=0;

        first = rules[i].rhs[0];

        if(rules[i].num!=0)

            second = rules[i].rhs[1];
    }

```

```

if(!isNonTerminal(first,noRules))
{
    NT = rules[i].lhs;

    for(j=0;j<noNT;j++)
        if(firsts[j].c==NT)
            break;

    if(!isInFirSet(NT,first,noNT))
    {
        index = firsts[j].len;
        firsts[j].set[index]=first;
        firsts[j].len++;
    }
}

}

/*printf("first set. . \n");
for(i=0;i<noNT;i++)
{
    printf("%c\tlen=%d\t%s\n",firsts[i].c,firsts[i].len,firsts[i].set);
}*/

for(i=0;i<noRules;i++)
{
    char first=0;

    char second=0;

    first = rules[i].rhs[0];

    if(rules[i].num!=0)
        second = rules[i].rhs[1];

```

```

    if(isNonTerminal(first,noRules))
    {
        NT1 = rules[i].lhs;

        add2first(NT1,noRules,noNT,rules[i].num,i);

        //printf("\n");

        //add2first(NT1,NT2,noRules,noNT);
    }
}

/*
for(i=0;i<noRules;i++)
{
    char first=0;

    first = rules[i].rhs[0];

    if(isNonTerminal(first,noRules))
    {
        NT1 = rules[i].lhs;

        NT2 = rules[i].rhs[0];

        printf("\n");

        add2first(NT1,NT2,noRules,noNT);
    }
}

*/

printf("first set. . . \n");

for(i=0;i<noNT;i++)
{
    printf("%c\tlen=%d\t%s\n",firsts[i].c,firsts[i].len,firsts[i].set);
}

//follow set calculation

```



```

add2follow('$',follows[0].c,noNT);

for(i=0;i<noNT;i++)
{
    NT1 = follows[i].c;

    findfollow(NT1,noRules,noNT);

    printf("\n");
}

printf("follow set. . .\n");

for(i=0;i<noNT;i++)
{
    printf("%c\tlen=%d\t%s\n",follows[i].c,follows[i].len,follows[i].set);
}

//terminals

printf("\n\n");

//making table for predictive parsing

int TABLE[noNT][noT];

int Tind,NTind;

for(i=0;i<noNT;i++)
    for(j=0;j<noT;j++)
        TABLE[i][j]=-1;

for(i=0;i<noRules;i++)
{
    NT = rules[i].lhs;

    NT1 = rules[i].rhs[0];

    if(!isNonTerminal(NT1,noRules))
    {

```

```

Tind = charIndexT(NT1,term,noT);

NTind = charIndexNT(NT,nonTerm,noT);

//printf("\n%c(%d),%c(%d),i=%d",NT,NTind,NT1,Tind,i);

if((Tind!=-1)&&(NTind!=-1))

    TABLE[NTind][Tind] = i;

if(NT1=='e')

{

    for(j=0;j<noNT;j++)

        if(follows[j].c==NT)

            break;

    for(k=0;k<follows[j].len;k++)

    {

        ch = follows[j].set[k];

        Tind = charIndexT(ch,term,noT);

        NTind = charIndexNT(NT,nonTerm,noT);

        //printf("\n%c,%c",NT,ch);

        if((Tind!=-1)&&(NTind!=-1)&&(TABLE[NTind][Tind]==-1))

            TABLE[NTind][Tind] = i;

    }

}

}

else if(isNonTerminal(NT1,noRules))

{

    for(j=0;j<noNT;j++)

        if(firsts[j].c==NT1)

            break;

    //printf("\n>>%c\n",firsts[j].c);

```

```

for(k=0;k<firsts[j].len;k++)
{
    ch = firsts[j].set[k];
    Tind = charIndexT(ch,term,noT);
    NTind = charIndexNT(NT,nonTerm,noT);
    //printf("\n%c,%c",NT,ch);
    if((Tind!=-1)&&(NTind!=-1))
        TABLE[NTind][Tind] = i;
}
if(isInFirSet(NT1,'e',noNT))
{
    for(j=0;j<noNT;j++)
        if(follows[j].c==NT)
            break;
    for(k=0;k<follows[j].len;k++)
    {
        ch = follows[j].set[k];
        Tind = charIndexT(ch,term,noT);
        NTind = charIndexNT(NT,nonTerm,noT);
        //printf("\n%c,%c",NT,ch);
        if((Tind!=-1)&&(NTind!=-1)&&(TABLE[NTind][Tind]==-1))
            TABLE[NTind][Tind] = i;
    }
}
}

printTable(TABLE,term,nonTerm);

```

```

printf("\nRule numbers reference. . .\n");

printf("Rule\tRule no.\n");

for(i=0;i<noRules;i++)

{

    printf("%c->",rules[i].lhs);

    for(j=0;j<(rules[i].num)+1;j++)

        printf("%c",rules[i].rhs[j]);

    printf("\t%d\n",i);

}

printf("\n");

//make parser

for(i=0;i<20;i++)

{

    STK[i]='\0';

}

//printStats();

char input[20];

printf("\nEnter input to check:");

scanf("%s",input);

printf("\n");

k=0;

char ip,stkTop,temp;

int rule_no;

int flag = 0;

ip = input[k];

push('E');

while(1)

```

```

{

    if(TOP==1&&ip=='$')
    {

        printf("\nSuccessfully parsed!");

        break;

    }

    else if((TOP==1&&ip!='$'))
    {

        printf("\nInput not successfully parsed!");

        break;

    }

    stkTop = STK[TOP];

    if(ip=='$')
    {

        for(i=0;i<noRules;i++)
        {

            if(rules[i].lhs==stkTop)
            {

                for(j=0;j<=rules[i].num;j++)
                {

                    if(rules[i].rhs[j]=='e')

                        flag = 1;

                }

            }

        }

        if(flag==0)
        {

```

```

        printf("\nInput not successfully parsed!");
        break;
    }
}

if(stkTop=='e')
{
    pop();
}

if(isNonTerminal(stkTop,noRules))
{
    Tind = charIndexT(ip,term,noT);
    NTind = charIndexNT(stkTop,nonTerm,noT);
    if(TABLE[NTind][Tind]!=-1)
    {
        rule_no = TABLE[NTind][Tind];
        pop();
    }
    else
        break;
    for(i=0;i<noRules;i++)
        if(i==rule_no)
            break;
    for(j=rules[i].num;j>=0;j--)
    {
        temp = rules[i].rhs[j];
        push(temp);
    }
}

```

```

    }

    else if(!isNonTerminal(stkTop,noRules))
    {
        if(stkTop==ip)
        {
            pop();
            ip = input[++k];
        }
    }

    printf("\nSTACK:%s<-TOP\t\tcurrent-Input-sym:%c",STK,ip);
}

printStats();

printf("\n");

return 0;
}

```

text.txt :

E->TA

A->+TA|^

T->FB

B->*FB|^

F->t|(E)

OUTPUT:

```

Non-Terminals are. . .
E      D      T      U      F

first set. . .
E      len=2    (i
D      len=2    +e
T      len=2    (i
U      len=2    *e
F      len=2    (i

follow set. . .
E      len=2    $)
D      len=2    $)
T      len=3    +$)
U      len=3    +$)
F      len=4    *+$)

Parsing Table

      +      *      (      )      i      $
E      -1     -1      0     -1      0     -1
D       1     -1     -1      2     -1      2
T      -1     -1      3     -1      3     -1
U       5      4     -1      5     -1      5
F      -1     -1      6     -1      7     -1

Rule numbers reference. . .
Rule      Rule no.
E->TD      0
D->+TD      1
D->e        2
T->FU       3
U->*FU      4
U->e        5
F->(E)      6
F->i        7

```


EXPERIMENT NO: 8

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

Aim: implementation of Intermediate code generation phase of compiler.**Theory:**

Three-address code (TAC) will be the intermediate representation used in our Decaf compiler. It is essentially a generic assembly language that falls in the lower-end of the mid-level IRs. Some variant of 2, 3 or 4 address code is fairly commonly used as an IR, since it maps well to most assembly languages.

Syntax

Each statement has the general form of:

$$\text{result} := \text{operand}_1 \text{ operator } \text{operand}_2$$

Such as:

$$x := y \text{ op } z$$

Where x , y and z are variables, constants, or temporary variables generated by the compiler. op represents any operator, e.g. an arithmetic operator.

Examples

```
int main(void)
{
    int i;
    int b[10];
    for(i=0; i<10;++i){
        b[i]= i*i;
    }
}
```

The preceding C program, translated into three-address code, might look something like the following:

```

i := 0          ; assignment

L1: if i >= 10 goto L2    ; conditional jump

t0 := i*i

t1 := &b        ; address-of operation

t2 := t1 + i    ; t2 holds the address of b[i]

    *t2 := t0    ; store through pointer

i := i + 1

goto L1

L2:

```

ALGORITHM:

Step1: Begin the program

Step 2: The expression is read from the file using a file pointer

Step 3: Each string is read and the total no. of strings in the file is calculated.

Step 4: Each string is compared with an operator; if any operator is seen then the previous string and next string are concatenated and stored in a first temporary value and the three address code expression is printed

Step 5: Suppose if another operand is seen then the first temporary value is concatenated to the next string using the operator and the expression is printed.

Step 6: The final temporary value is replaced to the left operand value.

Step 7: End the program

Industrial Application: It will use in many industries for different purpose

1. code-improving transformations

Conclusion:

Questions:

1. What are the various methods of implementing three address statements?

2. Define three address codes.

3. Generate the three address code for an expression $x := a + b * c + d$;

4. Generate the three address code for while $(i < 10) \{ x := 0; i := i + 1; \}$

5. What are the various types of intermediate code representation?

6. List the steps for implementing three address statements.

7. Explain file pointer.

8. Explain compiler steps.

Program:

```

#include<stdio.h>

#include<conio.h>

#include<string.h>

struct three
{
char data[10],temp[7];
}s[30];

void main()
{
    char *d1,*d2;
    int i=0,len=0;
    FILE *f1,*f2;

    clrscr();

    f1=fopen("exe1.txt","r");
    f2=fopen("exe2.txt","w");

    while(fscanf(f1,"%s",s[len].data)!=EOF)
    len++;

    for(i=0;i<=len;i++)
    {
        if(!strcmp(s[i].data,"=))
        {
            fprintf(f2,"\nLDA\t%s",s[i+1].data);

            if(!strcmp(s[i+2].data,"+"))
            fprintf(f2,"\nADD\t%s",s[i+3].data);

            if(!strcmp(s[i+2].data,"-"))
            fprintf(f2,"\nSUB\t%s",s[i+3].data);

```

```
        fprintf(f2, "\nSTA\t%s", s[i-1].data);  
    }  
}  
fclose(f1);  
fclose(f2);  
getch();  
}
```

Input: exe1.txt

```
t1 = in1 + in2  
t2 = t1 + in3  
t3 = t2 - in4  
out = t3
```

Output: exe2.txt

```
LDA  in1  
ADD  in2  
STA  t1  
LDA  t1  
ADD  in3  
STA  t2  
LDA  t2  
SUB  in4  
STA  t3  
LDA  t3  
STA  out
```

EXPERIMENT NO: 9

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

--

Aim: Implementation of code generation phase of compiler.

Theory:

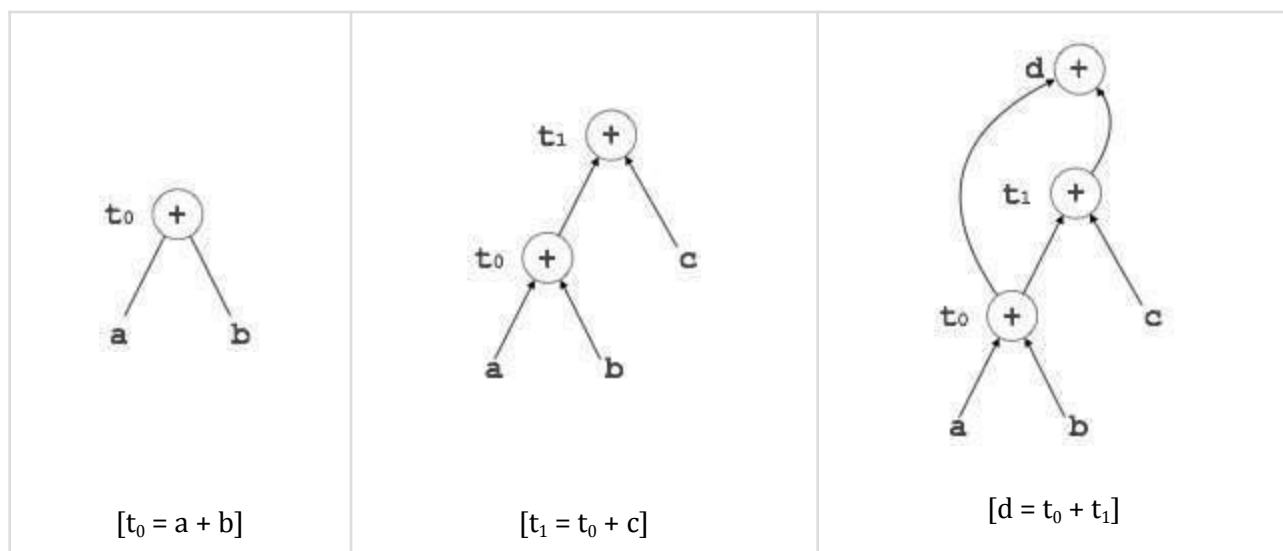
- 1) **Definition:** Code generation can be considered as the final phase of compilation. Through post code generation, optimization process can be applied on the code, but that can be seen as a part of code generation phase itself. The code generated by the compiler is an object code of some lower-level programming language, for example, assembly language.
- 2) **Directed Acyclic Graph:** Directed Acyclic Graph (DAG) is a tool that depicts the structure of basic blocks, helps to see the flow of values flowing among the basic blocks, and offers optimization too. DAG provides easy transformation on basic blocks. DAG can be understood here:
Leaf nodes represent identifiers, names or constants.
Interior nodes represent operators.
Interior nodes also represent the results of expressions or the identifiers/name where the values are to be stored or assigned.

Example:

$t0 = a + b$

$t1 = t0 + c$

$d = t0 + t1$



$[t_0 = a + b]$

$[t_1 = t_0 + c]$

$[d = t_0 + t_1]$

3) Peephole Optimization: This optimization technique works locally on the source code to transform it into an optimized code. By locally, we mean a small portion of the code block at hand. These methods can be applied on intermediate codes as well as on target codes. A bunch of statements is analyzed and are checked for the following possible optimization.

4) Redundant instruction elimination: At source code level, the following can be done by the user:

<pre>int add_ten(int x) { int y, z; y = 10; z = x + y; return z; }</pre>	<pre>int add_ten(int x) { int y; y = 10; y = x + y; return y; }</pre>	<pre>int add_ten(int x) { int y = 10; return x + y; }</pre>	<pre>int add_ten(int x) { return x + 10; }</pre>
--	---	---	--

At compilation level, the compiler searches for instructions redundant in nature. Multiple loading and storing of instructions may carry the same meaning even if some of them are removed. For example:

MOV x, R0

MOV R0, R1

We can delete the first instruction and re-write the sentence as:

```
MOV x, R1
```

- 5) Unreachable code:** Unreachable code is a part of the program code that is never accessed because of programming constructs. Programmers may have accidentally written a piece of code that can never be reached.

Example:

```
void add_ten(int x)
{
    return x + 10;

    printf("value of x is %d", x);
}
```

In this code segment, the printf statement will never be executed as the program control returns back before it can execute, hence printf can be removed.

- 6) Flow of control optimization;** There are instances in a code where the program control jumps back and forth without performing any significant task. These jumps can be removed. Consider the following chunk of code:

```
...
MOV R1, R2
GOTO L1
...
L1 : GOTO L2
L2 : INC R1
```

In this code, label L1 can be removed as it passes the control to L2. So instead of jumping to L1 and then to L2, the control can directly reach L2, as shown below:

```
...
MOV R1, R2
GOTO L2
...
L2 : INC R1
```

- 7) Algebraic expression simplification:** There are occasions where algebraic expressions can be made simple. For example, the expression $a = a + 0$ can be replaced by a itself and the expression $a = a + 1$ can simply be replaced by $\text{INC } a$.

8) Strength reduction

There are operations that consume more time and space. Their 'strength' can be reduced by replacing them with other operations that consume less time and space, but produce the same result.

For example, $x * 2$ can be replaced by $x \ll 1$, which involves only one left shift. Though the output of $a * a$ and a^2 is same, a^2 is much more efficient to implement.

9) Accessing machine instructions; The target machine can deploy more sophisticated instructions, which can have the capability to perform specific operations much efficiently. If the target code can accommodate those instructions directly, that will not only improve the quality of code, but also yield more efficient results.

10) Code Generator: A code generator is expected to have an understanding of the target machine's runtime environment and its instruction set. The code generator should take the following things into consideration to generate the code:

- **Target language:** The code generator has to be aware of the nature of the target language for which the code is to be transformed. That language may facilitate some machine-specific instructions to help the compiler generate the code in a more convenient way. The target machine can have either CISC or RISC processor architecture.
- **IR Type:** Intermediate representation has various forms. It can be in Abstract Syntax Tree (AST) structure, Reverse Polish Notation, or 3-address code.
- **Selection of instruction:** The code generator takes Intermediate Representation as input and converts (maps) it into target machine's instruction set. One representation can have many ways (instructions) to convert it, so it becomes the responsibility of the code generator to choose the appropriate instructions wisely.
- **Register allocation:** A program has a number of values to be maintained during the execution. The target machine's architecture may not allow all of the values to be kept in the CPU memory or registers. Code generator decides what values to keep in the registers. Also, it decides the registers to be used to keep these values.
- **Ordering of instructions:** At last, the code generator decides the order in which the instruction will be executed. It creates schedules for instructions to execute them.

11) Descriptors: The code generator has to track both the registers (for availability) and addresses (location of values) while generating the code. For both of them, the following two descriptors are used:

- **Register descriptor :** Register descriptor is used to inform the code generator about the availability of registers. Register descriptor keeps track of values stored in each register. Whenever a new register is required during code generation, this descriptor is consulted for register availability.
- **Address descriptor :** Values of the names (identifiers) used in the program might be stored at different locations while in execution. Address descriptors are used to keep track of memory locations where the values of identifiers are stored. These locations may include CPU registers, heaps, stacks, memory or a combination of the mentioned locations.

Code generator keeps both the descriptor updated in real-time. For a load statement, LD R1, x, the code generator:

updates the Register Descriptor R1 that has value of x and

updates the Address Descriptor (x) to show that one instance of x is in R1.

12) Code Generation: Basic blocks comprise of a sequence of three-address instructions. Code generator takes these sequence of instructions as input.

Note: If the value of a name is found at more than one place (register, cache, or memory), the register's value will be preferred over the cache and main memory. Likewise cache's value will be preferred over the main memory. Main memory is barely given any preference.

getReg: Code generator uses getReg function to determine the status of available registers and the location of name values. getReg works as follows:

If variable Y is already in register R, it uses that register.

Else if some register R is available, it uses that register.

Else if both the above options are not possible, it chooses a register that requires minimal number of load and store instructions.

For an instruction $x = y \text{ OP } z$, the code generator may perform the following actions. Let us assume that L is the location (preferably register) where the output of $y \text{ OP } z$ is to be saved:

Call function getReg, to decide the location of L.

Determine the present location (register or memory) of y by consulting the Address Descriptor of y. If y is not presently in register L, then generate the following instruction to copy the value of y to L:

MOV y', L

where y' represents the copied value of y .

Determine the present location of z using the same method used in step 2 for y and generate the following instruction:

OP z' , L

where z' represents the copied value of z .

Now L contains the value of y OP z , that is intended to be assigned to x . So, if L is a register, update its descriptor to indicate that it contains the value of x . Update the descriptor of x to indicate that it is stored at location L.

If y and z has no further use, they can be given back to the system.

Other code constructs like loops and conditional statements are transformed into assembly language in general assembly way.

Conclusion:

Questions:

1. What are Assembler Directives? Explain with example.

2. Explain various opcodes used in assembly language.

3. Explain pseudo codes

4. State the reason for assembler to be a multi-pass program.

1. Explain SPARC Assembler

PROGRAM:

```

#include<stdio.h>
void main()
{
    char stmt[4][6] = {"T=A-B"}, {"U=A-C"}, {"V=T+U"}, {"W=V+U"};
    struct code{
        char nemo[4];
        char op1[3];
        char op2[3];};
    struct code c[7];
    char add_dis[2][3],op;
    int i,cp=0,reg,j=0,flag,fnd_add;

    for(i=0;i<=3;i++){
        printf("\n%s",stmt[i]);
        op = stmt[i][3];
        flag = 0;
        switch(op){
            case '-':
                reg = getreg();
                strcpy(c[cp].nemo,"MOV");
                c[cp].op1[0] = stmt[i][2];
                c[cp].op1[1] = '\0';
                c[cp].op2[0] = 'R';
                c[cp].op2[1] = reg;
                c[cp].op2[2] = '\0';
                printf("\n%s\t%s\t%s",c[cp].nemo,c[cp].op1,c[cp].op2);

                cp++;
                strcpy(c[cp].nemo,"SUB");
                c[cp].op1[0] = stmt[i][4];
                c[cp].op1[1] = '\0';
                c[cp].op2[0] = 'R';
                c[cp].op2[1] = reg;
                c[cp].op2[2] = '\0';

                printf("\n%s\t%s\t%s",c[cp].nemo,c[cp].op1,c[cp].op2);
                //Assign Address Descriptor to variable on LHS of '=' sign
                add_dis[j][0] = stmt[i][0];
                printf("\nAddress Descriptor of ");
                printf("%c is ",add_dis[j][0]);
                add_dis[j][1] = 'R';
                printf("%c",add_dis[j][1]);
                add_dis[j][2] = reg;
                printf("%c",add_dis[j][2]);
                add_dis[j][3] = '\0';
                j++;
                cp++;
                break;
            case '+':
                strcpy(c[cp].nemo,"ADD");
                //search the address descriptor of second operand and store it as first
                opearnd in m/c instruction

```

```

        for(j=0;add_dis[j][0]!=stmt[i][4];j++);
        c[cp].op1[0] = 'R';
        c[cp].op1[1] = add_dis[j][2];
        c[cp].op1[2] = '\0';
        //Find the address descriptor of first operand and store it as second
opearnd in m/c instruction
        for(j=0;add_dis[j][0]!=stmt[i][2];j++);
        c[cp].op2[0] = 'R';
        c[cp].op2[1] = add_dis[j][2];
        c[cp].op2[2] = '\0';
        printf("\n%s\t%s\t%s",c[cp].nemo,c[cp].op1,c[cp].op2);
        //Assign Address Descriptor to variable on LHS of '=' sign
        add_dis[j][0] = stmt[i][0];
        printf("\nAddress Descriptor of %c is
%c%c",add_dis[j][0],add_dis[j][1],add_dis[j][2]);
        cp++;
        if(i==3){
            strcpy(c[cp].nemo,"MOV");
            c[cp].op1[0] = 'R';
            c[cp].op1[1] = add_dis[j][2];
            c[cp].op1[2] = '\0';
            c[cp].op2[0] = stmt[i][0];
            c[cp].op2[1] = '\0';
            printf("\n%s\t%s\t%s",c[cp].nemo,c[cp].op1,c[cp].op2);}

        break;  }}}

int getreg(){
    static int r=48;
    //printf("\n Register is %c",r);
    return r++;}

```

OUTPUT:

```

T=A-B
MOV      A      R0
SUB      B      R0
Address Descriptor of T is R0
U=A-C
MOV      A      R1
SUB      C      R1
Address Descriptor of U is R1
U=T+U
ADD      R1      R0
Address Descriptor of U is R0
W=U+U
ADD      R1      R0
Address Descriptor of W is R0
MOV      R0      W
Process returned 9 (0x9)    execution time : 0.016 s
Press any key to continue.

```

EXPERIMENT NO: 10

Name of the Student:- _____

Roll No. _____

Date of Practical Performed:- _____ Staff Signature with Date

& Marks

Aim: Study and implement experiments on LEX,YACC.

Theory:

Structure of a lex file

The structure of a lex file is intentionally similar to that of a yacc file; files are divided up into three sections, separated by lines that contain only two percent signs, as follows:

Definition section

%%

Rules section

%%

C code section

- The **definition** section is the place to define macros and to import header files written in C. It is also possible to write any C code here, which will be copied verbatim into the generated source file.
- The **rules** section is the most important section; it associates patterns with C statements. Patterns are simply regular expressions. When the lexer sees some text in the input matching a given pattern, it executes the associated C code. This is the basis of how lex operates.
- The **C code** section contains C statements and functions that are copied verbatim to the generated source file. These statements presumably contain code called by the rules in the rules section. In large programs it is more convenient to place this code in a separate file and link it in at compile time.

Yacc is the Utility which generates the function `yyparse`; which is indeed the Parser. Yacc describes a context free , LALR(1) grammar and supports both bottom-up and top-down parsing. The general format for the YACC file is very similar to that of the Lex file.

1. Declarations
2. Grammar Rules
3. Subroutines

Basic Specifications: Names refer to either tokens or nonterminal symbols. Yacc requires token names to be declared as such. In addition, for reasons discussed in Section 3, it is often

desirable to include the lexical analyzer as part of the specification file; it may be useful to include other programs as well. Thus, every specification file consists of three sections: the declarations, (grammar) rules, and programs. The sections are separated by double percent ``%'' marks. (The percent ``%'' is generally used in Yacc specifications as an escape character.)

In other words, a full specification file looks like

declarations

%%

rules

%%

programs

The declaration section may be empty. Moreover, if the programs section is omitted, the second %% mark may be omitted also;

thus, the smallest legal Yacc specification is %%

Conclusion:

.....

Questions:

1. What is the difference between LEX and YACC?

2. Explain lex and yacc tools.

3. Give the structure of the lex program?

4. Explain the structure of a yacc program.

5. What is lexical analyzer?

6. Why we have to include 'y.tab.h' in lex?

Program:**1)LEX:**

```

/*lex program to count number of words*/
%{
#include<stdio.h>
#include<string.h>
int i = 0;
%}

/* Rules Section*/
%%

([a-zA-Z0-9])* {i++;} /* Rule for counting number of words*/

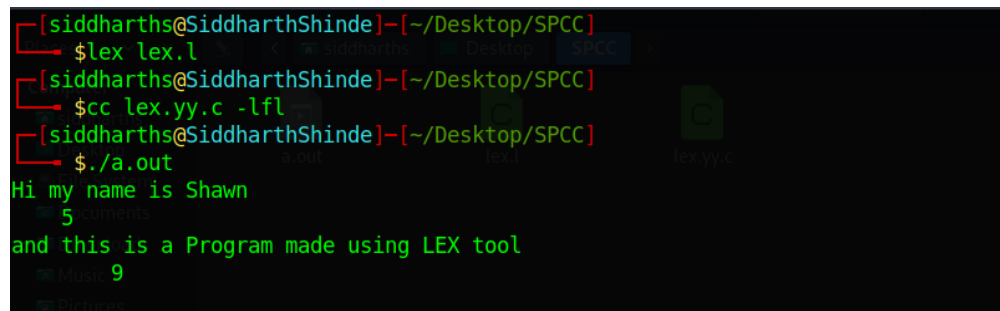
"\n" {printf("%d\n", i); i = 0;}

%%

int yywrap(void){}

int main()
{
    // The function that starts the analysis
    yylex();
    return 0;
}

```

Output:


```

[siddharths@SiddharthShinde]~/Desktop/SPCC
$lex lex.l
[siddharths@SiddharthShinde]~/Desktop/SPCC
$gcc lex.yy.c -lfl
[siddharths@SiddharthShinde]~/Desktop/SPCC
$./a.out
Hi my name is Shawn
and this is a Program made using LEX tool

```

2)YACC :

```

/**Program to count the number of characters, words, spaces, end of lines from the
given input line***/

```

```
#include <stdio.h>

#include <stdlib.h>

int main(){

FILE * file;

char path[100];

char ch;

int characters, words, lines;

/* Input path of files to merge to third file

*/ printf("Enter source file path: ");

scanf("%s", path);

/* Open source files in 'r' mode */

file = fopen(path, "r");

/* Check if file opened successfully */

if (file == NULL){

printf("\nUnable to open file.\n");

printf("Please check if file exists and you have read privilege.\n");

exit(EXIT_FAILURE);}

/*

* Logic to count characters, words and lines.

*/

characters = words = lines = 0;

while ((ch = fgetc(file)) != EOF){

characters++;

/* Check new line */

if (ch == '\n' || ch == '\0')

lines++;

/* Check words */
```

```

if (ch == ' ' || ch == '\t' || ch == '\n' || ch == '\0')
    words++;
}

/* Increment words and lines for last word */

if (characters > 0){
    words++;
    lines++;}

/* Print file statistics */

printf("\n");

printf("Total characters = %d\n", characters);

printf("Total words = %d\n", words);

printf("Total lines = %d\n", lines);

/* Close files to release resources */

fclose(file);

return 0;}

```

Input :

Yacc is the Utility which generates the function `yyparse`; which is indeed the Parser. Yacc describes a context free , LALR(1) grammar and supports both bottom-up and top-down

parsing. The general format for the YACC file is very similar to that of the Lex file.

Output :

```
C:\TURBOC3\BIN>TC
Enter source file path: C:\TURBOC3\BIN\text.txt

Total characters = 270
Total words = 45
Total lines = 3
```